PLANO DE DESENVOLVIMENTO

Sistema de Gestão de Estoque

Divisão de tarefas para 5 integrantes • Java + React • 4 semanas

Backend Java + Spring Boot	Frontend React + Vite	Banco de dados PostgreSQL

Objetivo

Organizar o desenvolvimento de um sistema integrado de gestão de estoque, distribuindo responsabilidades reais de frontend, backend, banco de dados, testes e integração entre cinco pessoas. O plano prioriza entregas demonstráveis e integração contínua ao longo de quatro semanas.

Princípio do projeto: todos devem entregar código funcional, testes e uma demonstração individual do próprio módulo.

1. Arquitetura e tecnologias

A aplicação será dividida em três camadas principais:

Camada	Tecnologia	Responsabilidade
Frontend	React + Vite, React Router, Axios	Telas, formulários, tabelas, gráficos, navegação e validações visuais.
Backend	Java + Spring Boot, Spring Web, Spring Data JPA, Spring Security, Bean Validation	API REST, regras de negócio, autenticação, validações e acesso aos dados.
Banco de dados	PostgreSQL + Flyway	Persistência relacional, chaves estrangeiras, transações e migrações versionadas.
Qualidade e colaboração	JUnit 5, Mockito, Git/GitHub, Docker Compose opcional	Testes, versionamento e ambiente de desenvolvimento reproduzível.

Fluxo recomendado: React → API REST do Spring Boot → Services → Repositories/JPA → PostgreSQL.

2. Divisão das tarefas entre os integrantes

Pessoa 1 — Produtos e categorias

- Backend: entidades Produto e Categoria, repositórios, serviços, controllers e DTOs.
- CRUD de produtos e categorias; validação de código, nome, preço e estoque mínimo.
- Frontend: listagem, cadastro, edição, filtros por nome/código/categoria e gestão de categorias.
- Entrega: módulo de produtos completo, integrado e com testes das regras principais.

Pessoa 2 — Entradas, saídas e estoque

- Backend: entidades Entrada e Saida; registro de movimentações e atualização automática do saldo.
- Bloquear saídas acima do estoque disponível e usar transações para manter consistência.
- Frontend: formulários de entrada/saída, seleção de produtos e fornecedores e histórico.
- Entrega: fluxo de movimentações testado, incluindo saldo atualizado e tentativa de saldo negativo.

Pessoa 3 — Dashboard e relatórios

- Backend: consultas de estoque atual, movimentações por período, estoque baixo e zerado.
- Frontend: indicadores, gráficos, filtros por produto/categoria/datas e exportação CSV.
- Conferir os resultados dos relatórios com os dados de movimentações cadastradas.
- Entrega: dashboard com dados reais e relatórios consistentes.

Pessoa 4 — Login, funcionários e permissões

- Backend: entidade Funcionario, autenticação com Spring Security, senhas com hash BCrypt e controle de acesso.
- CRUD de funcionários, perfis de acesso e desativação de contas.
- Frontend: login, cadastro/edição de funcionários e proteção das rotas por perfil.
- Entrega: login funcional e endpoints protegidos, com testes de permissões.

Pessoa 5 — Fornecedores, auditoria e integração

- Backend: CRUD de fornecedores, registro de auditoria de ações importantes e padronização de erros.
- Frontend: cadastro/consulta de fornecedores, layout principal, menu lateral, consultas rápidas e estados de erro/carregamento.

- Apoiar a integração dos módulos e criar testes de integração da API, sem substituir o desenvolvimento próprio.
- Entrega: fornecedores funcionando, interface integrada e fluxo entre módulos validado.

Equilíbrio da equipe

- A Pessoa 1 ajuda a definir o modelo inicial e os contratos da API.
- A Pessoa 2 entrega cedo o fluxo de movimentação para a Pessoa 3 desenvolver relatórios com dados reais.
- A Pessoa 4 combina a configuração de autenticação e permissões com a Pessoa 5.
- A Pessoa 3 prepara dados de demonstração e testa as consultas enquanto os outros módulos são concluídos.
- Cada integrante corrige os erros do próprio módulo; a Pessoa 5 coordena a integração, mas também entrega funcionalidades próprias.

3. Modelo de dados

O modelo recomendado tem nove tabelas principais. O escopo pode ser simplificado se o prazo apertar, mas os relacionamentos essenciais devem ser mantidos.

Tabela	Campos principais / finalidade
categorias	id, nome, descricao
produtos	id, codigo, nome, categoria_id, preco, quantidade, estoque_minimo, ativo
fornecedores	id, nome, documento, email, telefone, ativo
cargos	id, nome, descricao
funcionarios	id, nome, cargo_id, login, senha_hash, ativo
entradas	id, produto_id, fornecedor_id, funcionario_id, quantidade, data_entrada, observacao
saidas	id, produto_id, funcionario_id, quantidade, destinatario, data_saida, observacao
relatorios	id, tipo, data_geracao, funcionario_id, filtros — opcional, para histórico de relatórios
logs_auditoria	id, funcionario_id, acao, entidade, entidade_id, data_hora

Relacionamentos principais

- Uma categoria pode conter vários produtos; cada produto pertence a uma categoria.
- Um produto pode aparecer em muitas entradas e saídas.
- Um fornecedor pode estar associado a várias entradas.
- Um funcionário pode registrar várias movimentações, gerar relatórios e realizar ações auditadas.
- Cada funcionário está associado a um cargo.

Regras de banco e negócio

1. Cada tabela possui chave primária id, com geração automática.
2. Usar NUMERIC(12,2) para preço; evitar FLOAT para valores monetários.
3. A quantidade de uma movimentação deve ser um inteiro maior que zero.
4. A senha nunca deve ser armazenada em texto puro; usar senha_hash com BCrypt.
5. Produtos e funcionários com histórico devem ser desativados quando a exclusão física comprometer relacionamentos.
6. Entradas e saídas atualizam o saldo dentro de uma transação. O frontend não altera diretamente o saldo.
7. Usar Flyway para versionar a criação e as alterações das tabelas.

Decisão sobre o saldo: produtos.quantidade representa o estoque atual. As tabelas entradas e saidas mantêm o histórico que explica as alterações desse saldo.

4. Contratos da API REST

Responsável	Métodos e rotas sugeridas	Finalidade
P1	GET /api/produtos; GET /api/produtos/{id}; POST /api/produtos; PUT /api/produtos/{id}; DELETE /api/produtos/{id}	CRUD e consulta de produtos.
P1	/api/categorias	CRUD de categorias.
P2	GET/POST /api/entradas; GET/POST /api/saidas; GET /api/movimentacoes	Registrar e consultar movimentações.
P3	GET /api/relatorios/estoque; GET /api/relatorios/movimentacoes; GET /api/relatorios/estoque-baixo	Indicadores e relatórios.
P4	POST /api/auth/login; GET/POST /api/funcionarios; PUT/DELETE /api/funcionarios/{id}	Autenticação e gestão de usuários.
P5	/api/fornecedores; GET /api/dashboard	CRUD de fornecedores e indicadores gerais.

As rotas de CRUD devem seguir um padrão REST consistente. A equipe deve combinar os contratos na primeira semana e documentar os formatos de requisição e resposta.

Exemplo: cadastro de produto

```
POST /api/produtos
{
  "codigo": "PROD-001",
  "nome": "Teclado USB",
  "categoriaId": 1,
  "preco": 89.90,
  "estoqueMinimo": 10
}
```

O saldo inicial pode começar em zero; a quantidade disponível deve entrar por uma movimentação de entrada. A API deve validar os dados e usar códigos HTTP apropriados, como 201 para criação, 400 para dados inválidos, 401 para falta de autenticação e 404 para recurso inexistente.

5. Estrutura de pastas

```
gestao-estoque/
■■■ backend/
■   ■■■ src/main/java/br/com/estoque/
■   ■   ■■■ config/
■   ■   ■■■ security/
■   ■   ■■■ produto/
■   ■   ■■■ categoria/
■   ■   ■■■ entrada/
■   ■   ■■■ saida/
■   ■   ■■■ funcionario/
■   ■   ■■■ fornecedor/
■   ■   ■■■ relatorio/
■   ■   ■■■ auditoria/
■   ■   ■■■ common/
■   ■■■ src/main/resources/
■   ■   ■■■ application.yml
■   ■   ■■■ db/migration/
■   ■■■ src/test/
■   ■■■ pom.xml
■■■ frontend/
■   ■■■ src/
■   ■   ■■■ api/
■   ■   ■■■ components/
■   ■   ■■■ layouts/
■   ■   ■■■ pages/
■   ■   ■   ■■■ dashboard/
■   ■   ■   ■■■ produtos/
■   ■   ■   ■■■ categorias/
■   ■   ■   ■■■ entradas/
■   ■   ■   ■■■ saidas/
■   ■   ■   ■■■ relatorios/
■   ■   ■   ■■■ funcionarios/
■   ■   ■   ■■■ fornecedores/
■   ■   ■   ■■■ login/
■   ■   ■■■ routes/
■   ■   ■■■ hooks/
■   ■   ■■■ utils/
■   ■   ■■■ App.jsx
■   ■   ■■■ main.jsx
■   ■■■ package.json
■■■ docs/
■   ■■■ diagrama-er.png
■   ■■■ endpoints.md
■   ■■■ casos-de-teste.md
■■■ docker-compose.yml
■■■ README.md
```

Padrão obrigatório do backend

Controller → Service → Repository → PostgreSQL

- Controller: recebe requisições HTTP e devolve respostas.
- Service: contém as regras de negócio.
- Repository: acessa o banco com Spring Data JPA.
- DTO: define os dados aceitos e retornados pela API.
- Entity: representa as tabelas e seus relacionamentos.

Não colocar regras de negócio apenas nos controllers ou no React. A validação de estoque precisa existir no backend.

6. Cronograma de quatro semanas

Semana	Foco	Entregas
1 — Base comum	Modelagem, repositório, contratos, ambiente	Banco conectado, migrações funcionando, frontend e backend executando e primeiro endpoint acessível.
2 — Módulos	Desenvolvimento funcional em paralelo	Produtos, categorias, movimentações, login, fornecedores e primeiros relatórios funcionando.
3 — Integração	Regras, segurança, consultas e testes	Fluxo completo integrado; saldo validado; permissões e relatórios testados.
4 — Finalização	Correções, documentação e apresentação	Sistema reproduzível, README, diagrama, casos de teste e demonstração preparada.

Entregas por semana

- Semana 1: todos concordam com o modelo de dados, as rotas e o padrão de código. Cada pessoa entrega ao menos uma parte executável.
- Semana 2: concluir os fluxos principais de cada módulo.
- Semana 3: integrar os módulos e corrigir inconsistências.
- Semana 4: executar os testes finais, preparar documentação e ensaiar a apresentação.

7. Organização do GitHub

```
main
develop
feature/produtos
feature/movimentacoes
feature/relatorios
feature/autenticacao
feature/fornecedores
```

1. Cada pessoa cria sua branch a partir de develop.
2. Desenvolve e testa seu módulo com commits pequenos e claros.
3. Abre um pull request para revisão por outro integrante.
4. Após aprovação, integra em develop.
5. A equipe testa a versão integrada antes de atualizar main.

Arquivos compartilhados — segurança, entidades relacionadas, migrações e rotas globais — precisam de revisão coordenada para reduzir conflitos.

8. Critérios de aceitação por integrante

Pessoa	Demonstração obrigatória
P1	Cadastrar, editar, filtrar e desativar um produto.
P2	Registrar entrada e saída, mostrar mudança do saldo e bloquear saída sem estoque suficiente.
P3	Gerar relatório cujos totais correspondam às movimentações cadastradas.
P4	Fazer login, cadastrar funcionário e demonstrar uma restrição de permissão.
P5	Cadastrar fornecedor, consultar dados e demonstrar integração entre as telas.

Cada integrante também entrega código versionado, testes das funcionalidades de sua responsabilidade, evidências de execução e uma breve explicação das decisões técnicas do módulo.

9. Prioridades de escopo

Prioridade	Funcionalidades
Obrigatório	CRUD de produtos, categorias, entradas, saídas, funcionários e fornecedores; login; controle de estoque; consultas; dashboard básico; relatórios; validações e integração completa.
Se houver tempo	Exportação CSV, auditoria detalhada, gráficos adicionais, filtros avançados e paginação.
Versão futura	Leitor de código de barras, notificações por e-mail, múltiplos depósitos, integração fiscal e infraestrutura de produção.

10. Checklist antes de começar

- Definir o diagrama entidade-relacionamento.
- Combinar os contratos da API e os formatos JSON.
- Criar o repositório e a estrutura inicial.
- Definir regras de estoque, autenticação e permissões.
- Distribuir as tarefas da primeira semana.
- Garantir que todos consigam executar o projeto localmente.

Recomendação final: definam primeiro o modelo do banco e os contratos da API. Assim que esses contratos estiverem claros, os cinco integrantes podem desenvolver seus módulos em paralelo sem criar projetos incompatíveis.